🧾 FICHE MÉMO : Les algorithmes 🖥️

🎯 Objectif

Un **algorithme** est une **suite d'instructions claires et ordonnées** permettant de résoudre un problème ou d'accomplir une tâche.

> 💡 Penser à une recette de cuisine : chaque étape doit être précise pour obtenir le résultat attendu.

1️⃣ Caractéristiques d'un algorithme

- **Clair et précis** : chaque étape est compréhensible
- **Fini** : doit se terminer après un nombre fini d'étapes
- **Entrée / sortie** : reçoit des données (entrées) et produit un résultat (sortie)
- **Efficace** : résout le problème correctement et rapidement

2️⃣ Représentation d'un algorithme

a) En **pseudocode**

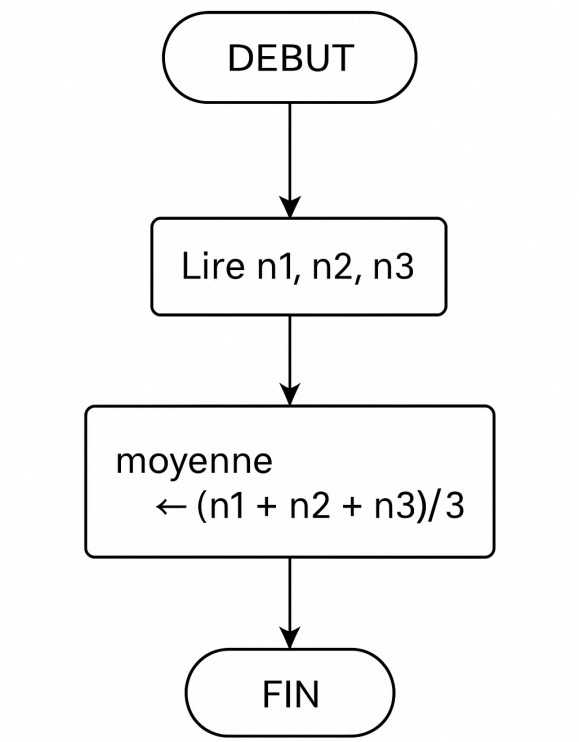
- Langage proche du français
- Permet de décrire la logique sans syntaxe exacte d'un langage

Exemple : Calcul de la moyenne de 3 nombres

```
DEBUT
   Lire nombre1, nombre2, nombre3
   moyenne ← (nombre1 + nombre2 + nombre3) / 3
   Afficher "La moyenne est ", moyenne
FIN
```

b) En **diagramme (organigramme)**

- Chaque étape est représentée par une **forme géométrique**
  - Ovale : début / fin
  - Rectangle : instruction / calcul
  - Losange : condition
- Permet de **visualiser le flux de l'algorithme**



3️⃣ Structures de base d'un algorithme

a) Séquence

Exécuter les instructions **une à une** dans l'ordre.

```
Lire a
Lire b
```

```
somme ← a + b
Afficher somme
```

b) Conditionnelle

Prendre **des décisions** selon une condition (`si ... alors ... sinon`).

```
Si age >= 18 Alors
   Afficher "Majeur"
Sinon
   Afficher "Mineur"
FinSi
```

c) Boucle (répétition)

Répéter **des instructions tant qu'une condition est vraie**.

```
Tant que compteur < 5 Faire
   Afficher compteur
   compteur ← compteur + 1
FinTantQue
```

4️⃣ Bonnes pratiques pour créer un algorithme

- Commencer par **définir le problème** clairement
- Déterminer **les données d'entrée et les résultats attendus**
- Diviser le problème en **étapes simples**
- Vérifier que **l'algorithme se termine**
- Tester avec **différents cas de figure**

5️⃣ Exemple pratique : Algorithme de tri par sélection

```
DEBUT
   Pour i de 0 à n-2
      min_index ← i
      Pour j de i+1 à n-1
```

```
            Si tableau[j] < tableau[min_index] Alors
                min_index ← j
            FinSi
        Echange tableau[i] et tableau[min_index]
    FinPour
FIN
```

- Trie un tableau de nombres du plus petit au plus grand
- Montre l'utilisation des **boucles imbriquées** et **conditions**

6️⃣ Résumé rapide

Concept	Description
Algorithme	Suite d'instructions pour résoudre un problème
Séquence	Instructions exécutées dans l'ordre
Condition	Choix selon un test (`si ... alors`)
Boucle	Répétition d'instructions
Entrée / sortie	Données reçues et résultats produits
Test et validation	Vérifier que l'algorithme fonctionne correctement

---

💡 Astuce :

Toujours commencer par **pseudocode ou organigramme** avant de coder dans un langage.

Ça aide à **visualiser la logique** et à **éviter les erreurs**.
